BÀI 4 – DANH SÁCH LIÊN KẾT ĐƠN – TRÊN ĐỐI TƯỢNG

I. MỤC TIÊU:

Khai báo danh sách liên kết đơn cho đối tượng

Viết chương trình thể hiện các tác vụ lên danh sách: cài đặt, khởi tạo, duyệt, thêm phần tử, loại bỏ phần tử, thay thế giá trị, ghép, trộn danh sách trên đối tượng.

II. TÓM TẮT LÝ THUYẾT

Nội dung	Ví dụ
1. Cấu trúc	

- Khai báo cấu trúc	`struct <tên kiểu cấu trúc>{`
	`<kiểu dữ liệu 1> tên thành phần 1;`
	`...`
	`<kiểu dữ liệu n> tên thành phần n;`
	`};`

- Một số lưu ý:	- Bắt buộc phải có ; sau dấu } - Tên của kiểu struct nên bắt đầu bằng ký tự hoa. - Các thành phần thuộc tính cùng kiểu dữ liệu nên cách nhau dấu , như các điểm toán, lý, hóa.
- Truy xuất biến kiểu cấu trúc	**- Đối với thành phần là 1 kiểu dữ liệu cơ bản:** <tên biến cấu trúc>.<tên thành phần> **- Đối với thành phần là 1 kiểu cấu trúc:** <tên biến cấu trúc 1>. <tên biến cấu trúc 2>.<tên thành phần>

- Ví dụ khai báo cấu trúc sinh viên gồm có các thành phần sau: masv, hotensv, namsinh, toan, ly, hoa, dtb.	`struct Student{`
	`  char id[10];`
	`  char fullName[30];`
	`  float score;`
	`};`

- **Khai báo cấu trúc danh sách sinh viên**	`typedef struct Node* NodePtr;`
	`struct Node{`
	`  Student data;`
	`  NodePtr link;`
	`};`

III. NỘI DUNG THỰC HÀNH

Câu 1: Đoạn mã sau dùng cấu trúc SinhVien gồm có **`id, fullName, score`** để nhập vào một danh sách sinh viên, và xuất danh sách sinh viên vừa nhập:

- Trong file **`thuvien_lab4.h`** như sau:

Dòng	Mã lệnh
1	`#define _CRT_SECURE_NO_WARNINGS`
2	`#include<iostream>`
3	`#include<iomanip>`
4	`#include"string.h"`
5	`using namespace std;`
6	`typedef struct Node* NodePtr;`
7	`struct Student{`
8	`  char id[10];`
9	`  char fullName[30];`
10	`  float score;`
11	`};`
12	`struct Node{`
13	`  Student data;`
14	`  NodePtr link;`
15	`};`
16	`void init(NodePtr& list);`
17	`int isEmpty(NodePtr list);`
18	`void freeList(NodePtr& list);`
19	`NodePtr createStudent(Student x);`
20	`NodePtr addFirst(NodePtr& list, Student x);`
21	`bool isDuplicate(NodePtr& list, char* id);`
22	`void createList(NodePtr &list);`
23	`void printList(NodePtr list);`

- Trong file **`caidat_lab4.cpp`** như sau:

Dòng	Mã lệnh
1	`#include"thuvien_lab4.h"`
2	`void init(NodePtr& list){`
3	`  list = NULL;`

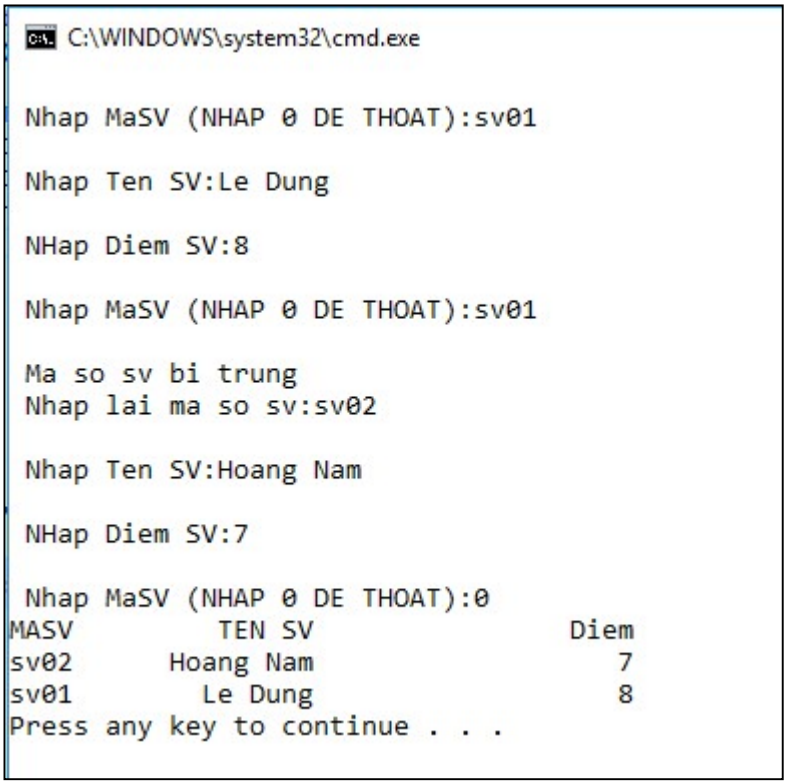
```
4   }
5   int isEmpty(NodePtr list){
6         return list == NULL ? 1 : 0;
7   }
8   void freeList(NodePtr& list){
9         NodePtr p = list;
10        while (p != NULL){
11              list = list->link;
12              delete p;
13              p = list;
14        }
15  }
16  NodePtr createStudent(Student x){
17        NodePtr p;
18        p = new Node;
19        p->data = x;
20        p->link = NULL;
21        return p;
22  }
23  //Ham Them 1 sinh vien
24  NodePtr addFirst(NodePtr &list, Student x)
25  {
26        NodePtr p = createStudent(x);
27        p->link = list;
28        list = p;
29        return p;
30  }
31  bool isDuplicate(NodePtr& list, char* id){
32        NodePtr p = list;
33        while (p != NULL){
34              if (strcmp(p->data.id, id) == 0)
35                    return true;
36              p = p->link;
37        }
38        return false;
39  }
40  //Nhap DSSV
41  void createList(NodePtr &list){
42        //init(list);
43        ////char  x[10];
44        Student student;
45        init(list);
46        do{
47              cout << "\n Enter id (enter 0 to exit):";
48              cin.getline(student.id, 10);
```

```
49              while (isDuplicate(list, student.id)){
50                    cout << "\n Id is duplicate";
51                    cout << "\n Enter id again,please:";
52                    cin.getline(student.id, 10);
53              }
54              if (strcmp(student.id, "0") == 0)  //NHAP MA SO = 0
55    DE THOAT
56                    break;
57              cout << "\n Enter fullname:";
58              cin.getline(student.fullName, 30);
59              cout << "\n Enter score:";
60              cin >> student.score;
61              cin.ignore();
62              addFirst(list, student);
63        } while (1);
64    }
65    void printList(NodePtr list){
66        cout << setw(0) << "Id" << setw(15) << "Fullname" <<
67    setw(20) << "Score" << endl;
68        while (list != NULL){
69              cout << setw(0) << list->data.id << setw(15) <<
70    list->data.fullName << setw(20) << list->data.score << endl;
71              list = list->link;
72        }
73    }
```

- Trong file **chuongtrinh_lab4.cpp** như sau:

Dòng	Mã lệnh
1	`#include"thuvien_lab4.h"`
2	`void main(){`
3	`NodePtr list;`
4	`createList(list);`
5	`printList(list);`
6	`freeList(list);`
7	`system("pause");`
8	`}`

Kết quả xuất ra ở đoạn mã trên như sau: (Hình ảnh chỉ mang tính chất minh họa)



Câu 2: Yêu cầu, sinh viên sử dụng đoạn mã trên để viết hàm thêm cuối sinh viên, gọi và chạy lại chương trình.

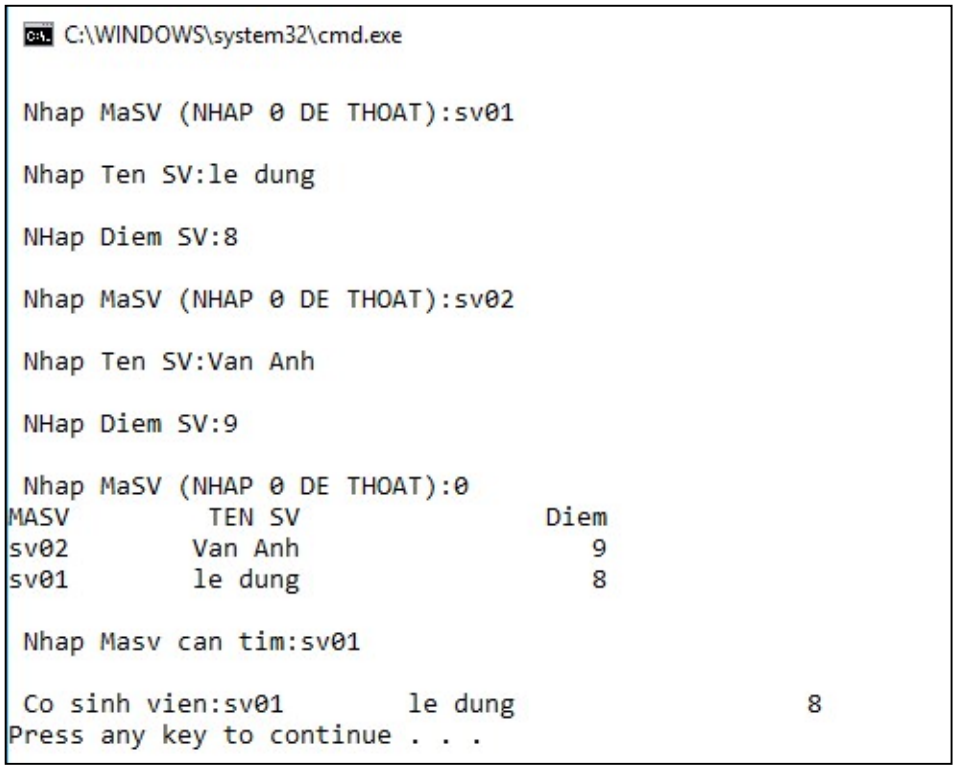
Câu 3: Cho hàm tìm kiếm một node trong DSLK:

Dòng	Mã lệnh
1	`NodePtr findXNode(NodePtr list, int x){`
2	`  NodePtr p = list;`
3	`  while (p != NULL){`
4	`  if (p->data == x)`
5	`  return p;`
6	`  p = p->link;`
7	`  }`
8	`  return p;`
9	`}`

Yêu cầu:

- Chỉnh sửa hàm **`findXNode`** ở trên, để xuất ra thông tin của sinh viên vừa tìm được dựa vào mã sinh viên nhập vào, có nguyên mẫu hàm như sau: **`NodePtr findStudentById(NodePtr list, Student x)`**

- Kết quả xuất ra như sau (Hình ảnh chỉ mang tính chất minh họa):



Câu 4: Đoạn mã sau đây là hàm xóa đầu (**deleteFirst**) của một danh sách.

Dòng	Mã lệnh
1	`void deleteFirst(NodePtr& list){`
2	`  NodePtr p;`
3	`  if (list != NULL){`
4	`  p = list;`
5	`  list = p->link;`
6	`  delete p;`
7	`  }`
8	`}`

Yêu cầu:

a/. Sinh viên hãy gọi hàm xóa đầu (**deleteFirst(NodePtr& list)**) vừa viết ở file cài đặt ở trên vào trong hàm **void main.** Xuất ra danh sách sinh viên còn lại sau khi gọi hàm xóa đầu.

b/. Sinh viên hãy viết thêm hàm xóa cuối (**deleteLast(NodePtr& list)**), sau đó gọi hàm xóa cuối vừa viết vào trong **void main** để thực thi chương trình.

Câu 5: Đoạn mã sau đây là hàm xóa một sinh viên khi biết mã sinh viên đó.

Dòng	Mã lệnh

```
1   void deleteStudentById(NodePtr& list, Student student){
2         if (strcmp(list->data.id, student.id) == 0){
3               NodePtr p = list;
4               list = list->link;
5               delete p;
6         }
7         else{
8               NodePtr p, before;
9               p = list;
10              before = list;
11              while (p != NULL &&strcmp(p->data.id, sv.id) != 0){
12                    before = p;
13                    p = p->link;
14              }
15              if (p != NULL){
16                    before->link = p->link;
17                    delete p;
18              }
19        }
20  }
```

Yêu cầu:

a/. Viết lời gọi hàm **`deleteStudentById`** ở trên trong hàm main.

b/. Viết hàm xóa tất cả các sinh viên có điểm <5 (**Bài tập nâng cao – về nhà**)

Gợi ý:

- Sử dụng hàm tìm kiếm những sinh viên có điểm dưới

- Viết hàm tìm kiếm những sinh viên có điểm dưới 5

- Viết hàm xóa tất cả các sinh viên có điểm dưới 5

IV. BÀI TẬP VỀ NHÀ

Câu 1: Sinh viên hãy viết hàm đảo ngược một DSLK có nguyên mẫu hàm như sau: `void reverseList(NotePtr &list).` viết thêm các lệnh cần thiết để chương trình có thể thực thi được.

Ví dụ : (Hình ảnh chỉ mang tính chất minh họa)

DS nhập vào là :

Mssv	Ten	Diem
A5130001	Nguyen Van Lai	8
A5130002	Tran Thi Mai	3
A5130003	Le Thi Dao	4

DS đảo ngược là :

Mssv	Ten	Diem
A5130003	Le Thi Dao	4
A5130002	Tran Thi Mai	3
A5130001	Nguyen Van Lai	8

Câu 2: Sinh viên hãy viết hàm **splitList()** để tách một danh sách sinh viên thành hai danh sách sinh viên trong đó :

- Một danh sách sinh viên chứa các sinh viên có điểm nhỏ hơn 5
- Một danh sách sinh viên chứa các điểm lớn hơn hoặc bằng 5

Viết thêm các hàm cần thiết để minh họa hàm vừa viết thực thi được.

Ví dụ : **(Hình ảnh chỉ mang tính chất minh họa)**

DS nhập vào là :

Mssv	Tên	Điểm
A5130001	Nguyen Van Lai	8
A5130002	Tran Thi Mai	3
A5130003	Le Thi Dao	4
A5130004	Ta Thi Cuc	5
A5130005	Ly Thi Hue	8
A5130006	Huynh Kiem Lan	9

Tách thành 2 DS :

- ***DSSV có điểm số nhỏ hơn 5 :***

```
A5130002      Tran Thi Mai      3
A5130003      Le Thi Dao        4
```

- ***DSSV có điểm số lớn hơn hoặc bằng 5 :***

```
A5130001      Nguyen Van Lai    8
A5130004      Ta Thi Cuc        5
A5130005      Ly Thi Hue        8
A5130006      Huynh Kien Lan    8
```

Câu 3. Nhập thêm vào cấu trúc sinh viên đã cho thông tin các điểm toán, lý, hóa. Yêu cầu tính điểm trung bình. Sắp xếp danh sách sinh viên theo chiều tăng dần của điểm trung bình.